AssemblyViz: A Web-Based Assembly Visualizer for HYMN and RISC-V

Nikita Aleksii, Robby Votta, Yurdanur Yolcu, Maria Fajardo
Department of Math and Computer Science, Davidson College

INTRODUCTION

- **simHYMN** is a desktop application used in Davidson College's Computer Organization course (CSC 250) to visualize an educational assembly language HYMN. However, its outdated design and limited functionality have necessitated modernization.
- **AssemblyViz** is a modernized counterpart to simHYMN that extends its core capabilities by providing the history of memory cells and debugging functionality. To offer deeper insight into assembly programming and CPU operation, AssemblyViz also introduces support for RISC-V instruction set architecture (ISA).

OBJECTIVES

- Replace simHYMN with a browser-based tool requiring no installation or login details.
- Support full HYMN simulation: 8 instructions, 3 registers, and 32 memory slots.
- Support full RISC-V simulation: 39 instructions and 6 pseudo-ops, across 32 registers.
- Enable step-by-step execution with forward and backward navigation.
- Display real-time register and memory updates after each instruction.
- Surface clear, line-specific error messages on assembly failure.

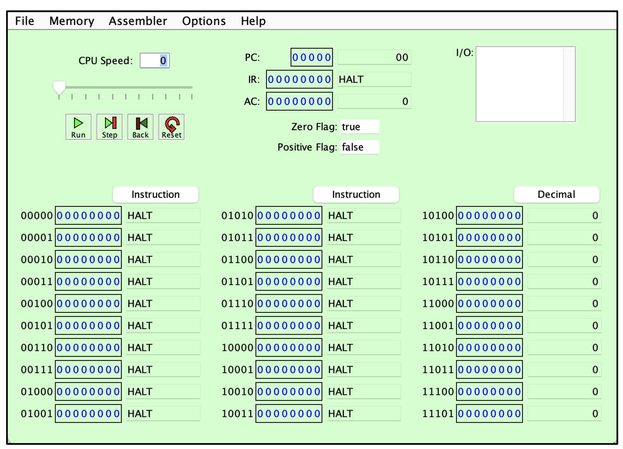

Figure 1. The original simHYMN interface, a desktop application displaying the CPU registers (PC, IR, AC), control buttons, and all 32 memory slots in binary. Its dated design and limited functionality motivated the development of AssemblyViz.

RESULTS

- Browser-based assembly visualization tool.
- Supports HYMN and RISC-V ISAs.
- Includes code editor, output space, registers, memory, and instruction encoding panel.
- The navigation bar supports play, pause, step back, step forward, and adjustable execution speed.
- Supports debugging that warns a user when an instruction or a register is not recognized.
- Supports a memory cell history that highlights changes to data in memory slots.

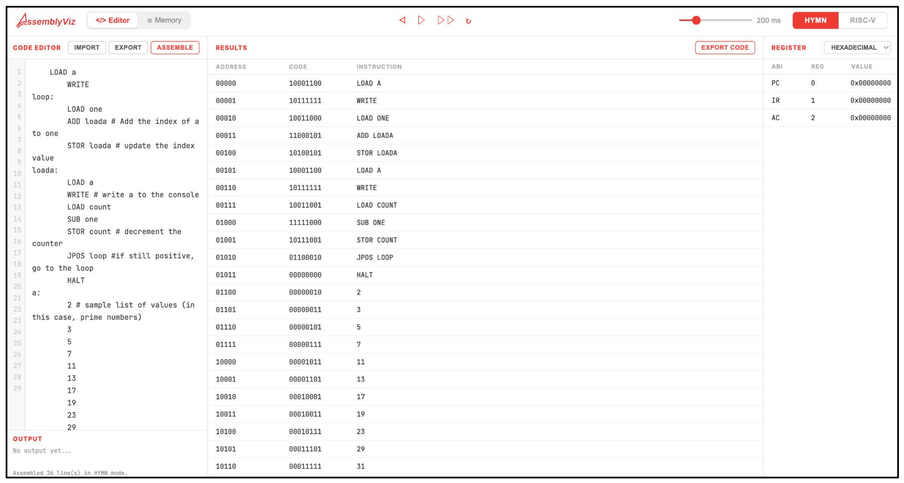

Figure 2. AssemblyViz running a HYMN program, showing the Editor view. The editor panel displays the code editor, assembled instruction encoding table, and register panel simultaneously.

To evaluate software performance, we measured the time it takes to execute RISC-V code of different sizes: 5, 10, 20, 50, and 100 lines of code.

- Register loop add – 20 iterations, constantly adding 1 to a variable.
- Sum 1 to 10 – continuously sums up integers from 1 to 10.
- Bubble sort – sorts a 5-element array.
- Fibonacci – computes first 20 terms of the Fibonacci sequence.
- Insertion sort – sorts a 10-element array.

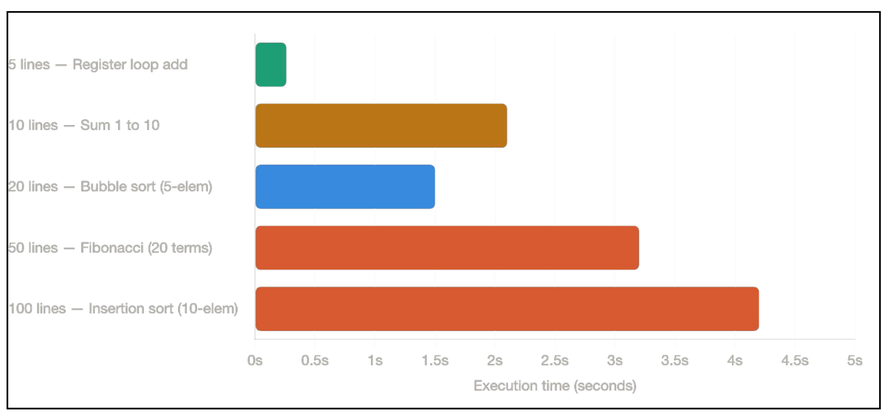

Figure 3. This figure shows the running times of different algorithms, written in RISC-V ISA. As the instructions count increases, it takes more time to execute them.

METHODS

- Followed an Agile/Scrum framework, enabling student feedback iteratively throughout development cycle.
- Python backend implementation as two independent simulation environments for HYMN and RISC-V, each following the same multi-stage pipeline:
  - User input parsed with regular expression (2-pass parser) into decoded instructions
  - Instructions loaded into memory and registers
  - Fetch-Decode-Execute cycle by stepping through line and running it
  - Debugger layer for breakpoint stepping. Snapshot returned to the frontend.
- Web application with React, Typescript, and Vite
- All data lives in one central place, the App component, and shared with four display components: Navbar, Code Editor, Results Panel, Register Panel.

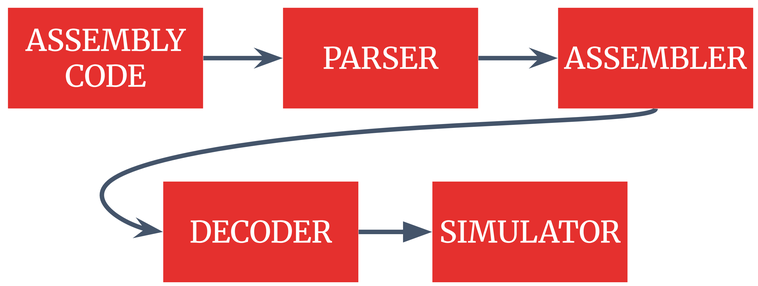

Figure 4. This graph illustrates the flow of assembly code through a multi-stage pipeline, beginning with parsing, followed by assembly, and concluding with two sequential decoding stages to produce the final output.

ACKNOWLEDGEMENTS

We thank Dr. Hammurabi Mendes for originating the project idea and for validating our implementation throughout development. We also thank Dr. Terrence Lim for the knowledge and guidance throughout the project lifecycle.